

Persistence

It is the ability to save and restore the state of a workflow over time.

Checkpointers LangGraph implements persistence using Checkpointers. A checkpointer divides our graph execution into "Supersteps" (a single step that may include parallel node executions). At every superstep, the checkpointer automatically ~~fetches that~~ saves both intermediate and final state values to a database or RAM.

Threads If multiple users are interacting to an application (like a Chatbot), the database needs a way to separate their memory. A ThreadId is assigned to uniquely identify a specific execution session or user. When we invoke the graph, we pass the Thread Id and LangGraph automatically fetches that specific conversation history from the checkpointer.

Four Core Benefits of Persistence

Adding Persistence unlocks four massive capabilities for Agentic Workflows:

- 1) Short-term memory $\Rightarrow$ It allows applications to remember past messages and enables users to "resume" an older conversation seamlessly.
- 2) Fault Tolerance $\Rightarrow$ If a workflow crashes mid-execution (e.g., an API goes down), it doesn't need to restart from beginning, because intermediate states are saved at every checkpoint, the workflow can resume exactly from the node where it failed.
- 3) Human-in-the-loop $\Rightarrow$ Allows a workflow to temporarily ~~execute~~ suspend execution to wait for a human's approval (e.g. approving a LinkedIn post before the AI publishes it). Because the state is saved permanently/persistently, the workflow can wait days for input from human without consuming compute resources.

4) Time Travel $\Rightarrow$ We can fetch the state of a past checkpoint to inspect it for debugging. Furthermore, we can "replay" the graph from that exact past checkpoint, or even manually update the state at that past checkpoint to create an alternate execution "fork" or branch.

Forking $\Rightarrow$ Forking is an advanced feature powered by Checkpointers (persistence) that falls under the broader category/capability known as Time Travel. It allows us to rewind our workflow to a past state, modify the data, and create an alternate execution timeline (a "branch" or "fork") without erasing or overwriting our original execution data.

Streaming $\Rightarrow$ Streaming means the LLM starts sending text tokens back to the UI the moment they are generated, rather than waiting for the entire response to finish generating.

Integrating Tools in LangGraph

To make an application fully "Agentic", it must be able to perform external actions like performing web search, fetching data from APIs, etc, rather than just conversing. In LangGraph, it is handled by two different components:

1. **ToolNode**: A prebuilt node provided by LangGraph (`langgraph.prebuilt.ToolNode`). Instead of writing custom logic, we pass a list of our tools to this node. It automatically listens for tool calls request from the LLM, routes to the request to the correct tool, executes it and returns the output.

2. **Tools Condition**: A pre-built conditional edge (`tools_condition`). It reads the LLM output and makes a routing decision: If the LLM requested an action, it routes the flow to the Tool Node. If the LLM just generated the conversational response, it routes the flow to the **END** node.

Workflow Architecture Rules:

⇒ The Cyclic Loop: We simply cannot route from the Chat Node → Tool Node → END. If we do this, the user will see raw JSON data outputted by the tool API, and multi-step tool reasoning becomes impossible. Instead, we must create a loop back to the Chat Node (Chat Node → Tool Node → Chat Node). This allows the LLM to read the raw JSON from the tool, understand it, and format a human-readable response before going to the END node.

Human In The Loop (HITL)

HITL is a design approach where a human actively participates at critical points in an Autonomous AI workflow to supervise, approve, correct, or guide the model's decisions.

Why HITL is essential?

1. Agent Imperfection: LLMs can hallucinate, misinterpret goals, or struggle with ambiguous requests (e.g. "book a flight for next Friday"). HITL allows the agent to pause and ask the user for clarification.

2. Accountability: AI cannot take legal or financial blame. For sensitive actions like sending emails on behalf of a company or processing payments, a human must provide final approval.

LangGraph Implementation

a) The interrupt function: Inside a sensitive node (like a "Purchase" node, the graph execution is paused using a function. The graph's current state is saved to the database (via Checkpoints), and a message is sent to front-end asking for user input.

b) The Command object: Once the human clicks "Yes" or "No" on the frontend, the graph is re-invoked. We pass the user's feedback to the backend using a Command object configured to resume execution. The graph wakes up from the exact node it paused at, reads the user's decision, and proceeds accordingly.

Alternative HITL methods: While `interrupt()` pauses execution inside a specific node, LangGraph also offers graph-level breakpoints. We can compile our graph with `interrupt_before = ["node name"]` or `interrupt_after = ["node name"]`. This pauses entire graph before or after a specific node runs, which is useful if we don't want to write explicit interrupt logic inside our node functions.